

CPJ-Cobranca AI - n8n

Manual tecnico e documentacao de arquitetura dos workflows



Desafio Tecnico Dev Pleno

Versao n8n da solucao de agentes de IA para apoio ao processo de desenvolvimento

Autor: Cledson Santos

Data: Junho de 2026

Versao: 1.0.0

Sumario

- 01.** Visao geral
- 02.** Acesso e setup
- 03.** Workflows e arquitetura
- 04.** Contratos HTTP
- 05.** Dados, cache e telemetria
- 06.** Deploy e operacao
- 07.** Validacao e troubleshooting

Visão geral

Este projeto é a versão n8n do case técnico CPJ-Cobranca AI. A ideia é demonstrar a mesma solução do `case-ts`, mas usando workflows exportáveis do n8n como unidade principal de entrega.

O foco do case é automatizar análises técnicas para um contexto de cobrança/recuperação de crédito:

- 1 revisar código com critérios de segurança, clareza, tratamento de erro e manutenção;
- 1 comparar tarefa e implementação para medir aderência;
- 1 gerar documentação técnica ou operacional;
- 1 gerar testes;
- 1 analisar Pull Requests do GitHub;
- 1 manter histórico auditável das execuções, com steps e telemetria.

Diferença para o case TypeScript

No `case-ts`, a solução é uma API Fastify/TypeScript com módulos, services, testes automatizados e painel web. Neste repositório, a API é composta por Webhook nodes do n8n. Cada workflow contém validação, chamada aos agentes, normalização da resposta e persistência.

Essa escolha mostra outra forma de entregar a mesma capacidade:

- 1 menor volume de código de aplicação;
- 1 fluxos visuais importáveis e editáveis no n8n;
- 1 integração direta com credenciais e nodes prontos;
- 1 persistência customizada em PostgreSQL para manter rastreabilidade;
- 1 deploy simples em VPS com Docker Compose.

Estado atual

Os workflows principais já estão exportados em `workflows/`:

- 1 `health`
- 1 `review`
- 1 `review/pull-request`
- 1 `compliance`

- 1 `document`
- 1 `tests`
- 1 `tests/pull-request`
- 1 `batch`
- 1 `history`
- 1 `analytics/usage`

Os fluxos de IA usam OpenRouter via LangChain nodes. Os fluxos utilitarios leem ou agregam dados do PostgreSQL.

Principios da implementacao

- 1 **Contratos parecidos com o case TS:** payloads e respostas seguem os mesmos nomes de campos principais.
- 1 **Cache por hash do payload:** requests repetidos reaproveitam a ultima resposta bem-sucedida do mesmo fluxo.
- 1 **Historico auditavel:** cada execucao grava entrada, saida, status, duracao, cache hit e steps.
- 1 **Telemetria:** os fluxos persistem modelo solicitado/usado, tokens estimados e custo estimado quando aplicavel.
- 1 **Portabilidade:** os workflows sao JSONs versionados, importaveis em qualquer instancia n8n com as credenciais equivalentes.

Limites e decisoes conscientes

- 1 Os Webhook nodes exportados nao possuem autenticacao propria; a protecao deve ser feita no proxy ou adicionada nos nodes antes de exposicao publica.
- 1 A telemetria de tokens/custos dos fluxos de codigo direto e estimada no workflow a partir do tamanho dos prompts/respostas.
- 1 O catalogo dinamico de prompts/modelos do `case-ts` nao foi reproduzido como CRUD separado; os workflows aceitam `model` e alguns aceitam `prompt_version`, mas os prompts estao embutidos no JSON exportado.
- 1 Nao ha painel web proprio; a operacao acontece pelo editor n8n, pelos endpoints e pelas consultas de historico/analytics.

Acesso e setup

URLs

Editor n8n:

- 1 Local: `http://localhost:5678`
- 1 Producao: `https://n8n.preambulo.cledson.com.br`

Webhooks em producao:

- 1 Base local: `http://localhost:5678/webhook`
- 1 Base producao: `https://n8n.preambulo.cledson.com.br/webhook`

Exemplo:

```
POST https://n8n.preambulo.cledson.com.br/webhook/api/v1/review
Content-Type: application/json
```

Durante execucao de teste pelo editor, o n8n pode gerar URLs temporarias com `/webhook-test/...`. Para consumo externo estavel, ative o workflow e use `/webhook/...`.

Subindo localmente

```
cp .env.example .env
docker compose up -d
```

Antes de subir em um ambiente compartilhado, ajuste no `.env`:

- 1 `POSTGRES_PASSWORD`
- 1 `N8N_BASIC_AUTH_USER`
- 1 `N8N_BASIC_AUTH_PASSWORD`
- 1 `N8N_ENCRYPTION_KEY`
- 1 `OPENROUTER_API_KEY`

Depois confira:

```
docker compose ps
docker compose logs -f n8n
```

Credenciais no n8n

Crie as credenciais com os mesmos nomes usados pelos workflows exportados:

Nome	Tipo esperado	Uso
case-n8n Postgres	PostgreSQL	Cache, historico, steps, batch e analytics
OpenRouter account	OpenRouter/LangChain	Chamadas LLM
GitHub account	GitHub API	Fluxos de Pull Request
Jira account	Jira API	Critérios Jira opcionais no review de PR

Para o PostgreSQL no Docker Compose:

- 1 host: postgres
- 1 port: 5432
- 1 database: valor de POSTGRES_DB
- 1 user: valor de POSTGRES_USER
- 1 password: valor de POSTGRES_PASSWORD

Importando workflows

- 1 Acesse o editor n8n.
- 2 Crie as credenciais acima.
- 3 Importe os arquivos JSON de workflows/.
- 4 Confirme se os nodes de credencial apontam para as credenciais corretas.
- 5 Salve e ative cada workflow que sera consumido.
- 6 Use as URLs de producao exibidas pelo Webhook node, normalmente em /webhook/...

Ordem sugerida:

- 1 health.json
- 2 review.json
- 3 compliance.json
- 4 document.json
- 5 tests.json
- 6 pull-request-review.json

- 7 `pull-request-tests.json`
- 8 `history-list.json`
- 9 `history-detail.json`
- 10 `analytics-usage.json`
- 11 `batch.json`

O `batch` chama internamente `/webhook/api/v1/review`, `/webhook/api/v1/compliance`, `/webhook/api/v1/document` e `/webhook/api/v1/tests`; por isso esses workflows precisam estar ativos antes do batch.

Testando

Use `examples/case.http` ou qualquer cliente HTTP.

```
POST http://localhost:5678/webhook/api/v1/tests
Content-Type: application/json

{
  "code": "export function calcularJurosSimples(principal, taxaMensal, meses) { return
    principal * (1 + taxaMensal * meses); }",
  "language": "typescript",
  "test_framework": "vitest"
}
```

Autenticacao dos webhooks

O Basic Auth configurado por `N8N_BASIC_AUTH_ACTIVE=true` protege o editor n8n. Os Webhook nodes deste repositorio nao declaram autenticacao propria no JSON exportado.

Para ambiente publico, use uma das estrategias:

- 1 proteger `/webhook/*` no Nginx com Basic Auth ou outra politica;
- 1 adicionar autenticacao diretamente nos Webhook nodes;
- 1 expor os webhooks apenas em rede privada/VPN;
- 1 colocar um gateway autenticado na frente do n8n.

Workflows e arquitetura

Visao de arquitetura

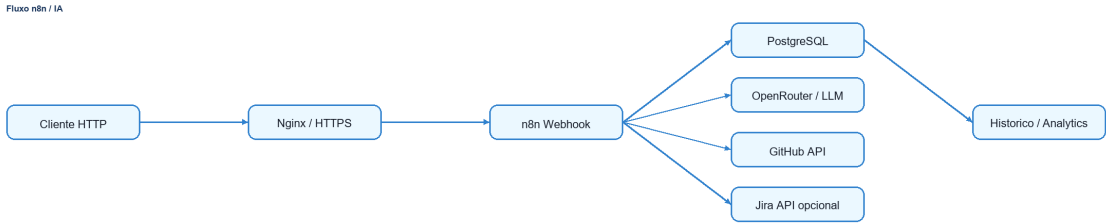


Diagrama do fluxo documentado.

O n8n é o orquestrador principal. Cada workflow recebe a chamada HTTP, valida payload, procura cache no PostgreSQL, executa tools determinísticas e/ou agentes, normaliza a resposta e persiste a execução.

Inventario

Arquivo	Workflow	Endpoint	Papel
health.json	Health Check	GET /webhook/health	Verificacao simples de vida
review.json	Professional Code Review AI Agents	POST /webhook/api/v1/review	Review multi-agente de código
pull-request-review.json	Pull Request Review AI Agents	POST /webhook/api/v1/review/pull-request	Review de PR no GitHub com Jira opcional
compliance.json	Compliance AI Agent	POST /webhook/api/v1/compliance	Aderencia entre tarefa e código
document.json	Document AI Agent	POST /webhook/api/v1/document	Documentacao tecnica/operacional
tests.json	Tests AI Agent	POST /webhook/api/v1/tests	Geracao de testes por código
pull-request-tests.json	Pull Request Tests AI Agent	POST /webhook/api/v1/tests/pull-request	Geracao de testes baseada em PR
batch.json	Batch Flow Orchestrator	POST /webhook/api/v1/batch	Orquestracao sequencial de fluxos
history-list.json	History List API	GET /webhook/api/v1/history	Lista execucoes
history-detail.json	History Detail	GET /webhook/api/v1/history/:id	Detalhe de execucao
analytics-usage.json	Analytics Usage	GET /webhook/api/v1/analytics/usage	Agregacao de uso, tokens e custos

Fluxo padrao dos agentes

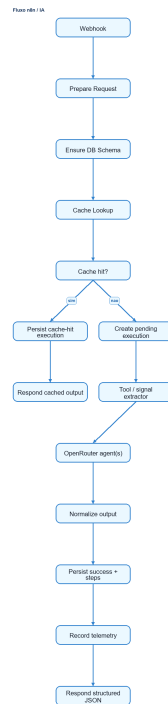


Diagrama do fluxo documentado.

Review de codigo

O `review.json` e o workflow mais completo. Ele roteia por linguagem (`typescript`, `javascript`, `python`, `php`), executa checks determinísticos simples e chama agentes especialistas:

- 1 Naming Clarity Agent
- 1 Error Handling Agent
- 1 Resource Leak Agent
- 1 Complexity Agent
- 1 Security Agent
- 1 Review Aggregator Agent

O agregador consolida achados repetidos e devolve o contrato publico:

- 1 `overall_quality`
- 1 `score`
- 1 `issues`
- 1 `positives`
- 1 `summary`

Compliance, documentacao e testes

Os workflows `compliance`, `document` e `tests` seguem o mesmo padrao:

- 1 validam campos obrigatorios;
- 1 aceitam `model` opcional, com fallback para `openai/gpt-4o-mini`;
- 1 aceitam `prompt_version` positivo quando enviado;
- 1 usam um node de extracao deterministica antes do agente;
- 1 normalizam a saida para um JSON estavel;
- 1 persistem execucao, steps e telemetria.

Pull Requests

Os fluxos de PR usam GitHub API para buscar:

- 1 metadados do PR;
- 1 diff;
- 1 arquivos alterados.

O review de PR pode consultar Jira quando o request enviar `jira_issue_key` e `jira_base_url`. Sem Jira, a secao correspondente e marcada como ignorada/skipped.

Batch

O workflow `batch.json` recebe uma lista de itens e executa os fluxos individuais via HTTP interno:

- 1 `review`
- 1 `compliance`
- 1 `document`
- 1 `tests`

Ele aceita `continue_on_error`; quando `false`, a execucao para no primeiro erro. O resumo do batch e salvo em `batch_executions`.

Contratos HTTP

Base de produção padrão:

```
https://n8n.preambulo.cledson.com.br/webhook
```

As rotas abaixo são relativas a essa base. Por exemplo, **POST /api/v1/review** significa **POST https://n8n.preambulo.cledson.com.br/webhook/api/v1/review**.

Linguagens suportadas

Os fluxos de código aceitam:

- 1 **typescript**
- 1 **javascript**
- 1 **python**
- 1 **php**

POST /api/v1/review

Request:

```
{
  "code": "function sum(a, b) { return a + b; }",
  "language": "javascript",
  "context": "Contexto opcional",
  "model": "openai/gpt-4o-mini",
  "prompt_version": 1
}
```

Campos obrigatórios: **code**, **language**.

Response:

```
{
  "overall_quality": "needs_improvement",
  "score": 7,
  "issues": [
    {
      "severity": "medium",
      "line_hint": "linha 1",
      "description": "Descricao objetiva do problema.",
      "suggestion": "Sugestao acionavel."
    }
  ],
  "positives": ["Ponto positivo observado."],
  "summary": "Resumo do review."
}
```

POST /api/v1/review/pull-request

Request:

```
{
  "github_pull_request_url": "https://github.com/org/repo/pull/123",
  "base_branch": "main",
  "jira_issue_key": "CPJ-123",
  "jira_base_url": "https://empresa.atlassian.net",
  "model": "openai/gpt-4o-mini",
  "prompt_version": 1
}
```

Obrigatorio: `github_pull_request_url`.

`jira_issue_key` e `jira_base_url` sao opcionais em conjunto. Quando `jira_issue_key` for enviado, `jira_base_url` precisa ser HTTPS.

Response:

```
{
  "verdict": "needs_attention",
  "score": 75,
  "summary": "Resumo consolidado da analise do PR.",
  "pull_request": {
    "owner": "org",
    "repo": "repo",
    "number": 123,
    "title": "Pull request #123",
    "base_branch": "main",
    "head_sha": "abc123",
    "changed_files": 4
  },
  "jira": null,
  "sections": {
    "code_standard": { "status": "warning", "findings": [] },
    "project_consistency": { "status": "warning", "findings": [] },
    "security": { "status": "warning", "findings": [] },
    "jira_criteria": { "status": "skipped", "criteria": [] }
  },
  "positives": [],
  "recommendations": []
}
```

POST /api/v1/compliance

Request:

```
{
  "task_description": "Registrar tentativa de contato com devedor.",
  "code": "app.post('/contatos', async (req, res) => res.status(201).json({ ok: true }));",
  "language": "javascript",
  "model": "openai/gpt-4o-mini"
}
```

Campos obrigatorios: **task_description**, **code**, **language**.

Response:

```
{
  "compliant": false,
  "compliance_score": 60,
  "covered_requirements": [],
  "missing_requirements": ["Nao ha evidencia de persistencia da tentativa de contato."],
  "partial_requirements": [],
  "verdict": "A implementacao cobre apenas parte da tarefa."
}
```

POST /api/v1/document

Request:

```
{
  "code": "export function charge(amount: number) { return amount > 0; }",
  "language": "typescript",
  "doc_type": "technical",
  "model": "openai/gpt-4o-mini"
}
```

Campos obrigatorios: **code**, **language**, **doc_type**.

doc_type aceita **technical** ou **operational**.

Response:

```
{
  "doc_type": "technical",
  "title": "Serviço de cobrança",
  "description": "Descrição da rotina analisada.",
  "inputs": [
    {
      "name": "amount",
      "type": "number",
      "description": "Valor da cobrança."
    }
  ],
  "outputs": [
    {
      "name": "return",
      "type": "boolean",
      "description": "Indica se o valor é positivo."
    }
  ],
  "side_effects": [],
  "usage_example": "charge(100)",
  "notes": null
}
```

POST /api/v1/tests

Request:

```
{
  "code": "export function charge(amount: number) { return amount > 0; }",
  "language": "typescript",
  "test_framework": "vitest",
  "model": "openai/gpt-4o-mini"
}
```

Campos obrigatórios: **code**, **language**, **test_framework**.

Response:

```
{
  "framework": "vitest",
  "test_file": "import { expect, it } from 'vitest';\n\nit('valida valor positivo', () => {\n  expect(charge(100)).toBe(true);\n});",
  "test_cases": [
    {
      "name": "valida valor positivo",
      "type": "happy_path",
      "description": "Cobre o caminho principal."
    }
  ],
  "coverage_hints": []
}
```

POST /api/v1/tests/pull-request

Request:

```
{
  "github_pull_request_url": "https://github.com/org/repo/pull/123",
  "base_branch": "main",
  "test_framework": "vitest",
  "model": "openai/gpt-4o-mini"
}
```

Campos obrigatórios: `github_pull_request_url`, `test_framework`.

Response: mesmo contrato do `/api/v1/tests`. O campo `test_file` deve conter o código-fonte completo do teste, não apenas um caminho.

POST /api/v1/batch

Request:

```
{
  "continue_on_error": true,
  "notify": false,
  "items": [
    {
      "flow_type": "review",
      "payload": {
        "code": "function sum(a, b) { return a + b; }",
        "language": "javascript"
      }
    },
    {
      "flow_type": "tests",
      "payload": {
        "code": "export function charge(amount: number) { return amount > 0; }",
        "language": "typescript",
        "test_framework": "vitest"
      }
    }
  ]
}
```

`flow_type` aceita `review`, `compliance`, `document` e `tests`.

Response:

```
{
  "batch_id": "uuid",
  "status": "success",
  "results": [
    {
      "index": 0,
      "flow_type": "review",
      "status": "success",
      "output": {}
    }
  ]
}
```

GET /api/v1/history

Query params:

- 1 **limit**: inteiro de 1 a 100, default **20**
- 1 **cursor**: id da ultima execucao da pagina anterior
- 1 **flow_type**: **review**, **compliance**, **document**, **tests**, **batch**, **pull_request_review**, **pull_request_tests**
- 1 **status**: **pending**, **success**, **failed**
- 1 **model**: modelo solicitado
- 1 **from**: data inicial
- 1 **to**: data final
- 1 **cache_hit**: **true** ou **false**

Response:

```
{
  "items": [],
  "page": {
    "limit": 20,
    "next_cursor": null
  }
}
```

GET /api/v1/history/:id

Retorna input, output, telemetria e steps de uma execucao. IDs invalidos retornam erro **bad_request**; IDs validos nao encontrados retornam **not_found**.

GET /api/v1/analytics/usage

Query params opcionais:

- 1 **flow_type**
- 1 **model**
- 1 **from**
- 1 **to**

Response:


```
{
  "totals": {
    "executions": 0,
    "successful": 0,
    "failed": 0,
    "cache_hits": 0,
    "prompt_tokens": 0,
    "completion_tokens": 0,
    "total_tokens": 0,
    "cache_read_tokens": 0,
    "cost_total_usd": 0,
    "cost_input_usd": 0,
    "cost_output_usd": 0,
    "average_duration_ms": 0
  },
  "by_day": [],
  "by_flow": [],
  "by_model": []
}
```

Dados, cache e telemetria

Banco de dados

O PostgreSQL é usado pelo próprio n8n e também por tabelas customizadas do case.

O bootstrap fica em:

```
infra/sql/init/001_create_agent_runs.sql
```

Tabelas principais:

Tabela	Papel
<code>executions</code>	Registro principal de cada chamada de fluxo
<code>execution_steps</code>	Passos internos da execução, como cache, tools e agentes
<code>execution_telemetry</code>	Modelo, tokens e custo estimado por execução
<code>batch_executions</code>	Resumo das execuções em lote
<code>agent_runs</code>	Compatibilidade com uma versão anterior do bootstrap

Modelo de execução

Cada fluxo de IA cria uma linha em `executions` com:

- 1 `flow_type`
- 1 `status`
- 1 `input_payload`
- 1 `output_payload`
- 1 `duration_ms`
- 1 `request_hash`
- 1 `cache_hit`
- 1 `source_execution_id`
- 1 `error_message`

Quando há cache hit, o workflow cria uma nova execução marcada como `cache_hit=true` e aponta `source_execution_id` para a execução original.

Cache

Os fluxos `review`, `compliance`, `document`, `tests`, `pull_request_review` e `pull_request_tests` calculam hash do payload normalizado.

Processo:

- 1 normaliza campos relevantes do request;
- 2 calcula `request_hash`;
- 3 busca a ultima execucao `success` do mesmo `flow_type` e hash;
- 4 se existir, retorna `output_payload` persistido;
- 5 se nao existir, cria execucao `pending` e roda os agentes.

Esse cache evita custo repetido em requests identicos e deixa a origem rastreavel pelo `source_execution_id`.

Steps

Os fluxos gravam steps para explicar o que aconteceu internamente.

Exemplos:

- 1 `cache_lookup`
- 1 `language_router`
- 1 `deterministic_tools`
- 1 `requirements_extractor`
- 1 `document_signal_extractor`
- 1 `tests_signal_extractor`
- 1 `review_aggregator_agent`
- 1 `compliance_agent`
- 1 `document_agent`
- 1 `tests_agent`

Cada step armazena tipo (`kind`), status, payload de entrada/saida, duracao e erro quando houver.

Telemetria

A tabela `execution_telemetry` guarda:

```
1 provider
1 model_requested
1 model_used
1 prompt_tokens
1 completion_tokens
1 total_tokens
1 cost_usd
1 input_cost_usd
1 output_cost_usd
1 cache_read_tokens
```

Nos fluxos de código direto, tokens e custos são estimados no próprio workflow. A estimativa atual usa uma aproximação simples de caracteres por token e os valores de referência usados no case para `openai/gpt-4o-mini`.

Nos fluxos de Pull Request, a telemetria registra provedor/modelo e pode ser expandida para persistir tokens reais quando a integração do node fornecer essa informação diretamente.

Historico

`GET /webhook/api/v1/history` lista execuções com filtros por:

```
1 fluxo;
1 status;
1 modelo;
1 intervalo de datas;
1 cache hit;
1 paginação por cursor.
```

`GET /webhook/api/v1/history/:id` retorna o detalhe da execução com input, output, telemetry e steps.

Analytics

`GET /webhook/api/v1/analytics/usage` agrega:

- 1 total de execuções;
- 1 sucesso/falha;
- 1 cache hits;
- 1 tokens;
- 1 custos;
- 1 duração média;
- 1 agrupamento por dia;
- 1 agrupamento por fluxo;
- 1 agrupamento por modelo.

Filtros opcionais:

- 1 `flow_type`
- 1 `model`
- 1 `from`
- 1 `to`

Deploy e operação

Runtime

O `docker-compose.yml` sobre:

- 1 `postgres`: PostgreSQL 16 Alpine;
- 1 `n8n`: imagem oficial `docker.n8n.io/n8nio/n8n:latest`.

O n8n usa PostgreSQL como banco interno e monta `./n8n` em `/home/node/.n8n` para persistir configurações locais, credenciais criptografadas e estado do editor.

Variáveis de ambiente

Principais variáveis:

Variável	Uso
<code>N8N_HOST_PORT</code>	Porta local exposta em <code>127.0.0.1</code>
<code>POSTGRES_DB</code>	Banco PostgreSQL
<code>POSTGRES_USER</code>	Usuário PostgreSQL
<code>POSTGRES_PASSWORD</code>	Senha PostgreSQL
<code>N8N_HOST</code>	Host público
<code>N8N_PROTOCOL</code>	<code>http</code> ou <code>https</code>
<code>WEBHOOK_URL</code>	Base pública usada pelos webhooks
<code>N8N_EDITOR_BASE_URL</code>	Base pública do editor
<code>N8N_BASIC_AUTH_ACTIVE</code>	Liga Basic Auth no editor
<code>N8N_BASIC_AUTH_USER</code>	Usuário do editor
<code>N8N_BASIC_AUTH_PASSWORD</code>	Senha do editor
<code>N8N_ENCRYPTION_KEY</code>	Chave de criptografia das credenciais
<code>OPENROUTER_API_KEY</code>	Chave do provedor LLM
<code>LANGSMITH_*</code> / <code>LANGCHAIN_*</code>	Tracing opcional

`N8N_ENCRYPTION_KEY` deve ser estável entre deploys. Trocar essa chave sem migrar credenciais pode impedir o n8n de descriptografar credenciais antigas.

Deploy por GitHub Actions

O arquivo `.github/workflows/deploy-production.yml` roda no push para `main`.

Fluxo:

- 1 Faz checkout do repositório.
- 2 Valida arquivos obrigatórios.
- 3 Conecta na VPS por SSH.
- 4 Clona ou atualiza o repositório no diretório de produção.
- 5 Gera `.env.production` a partir de secrets.
- 6 Ajusta permissão da pasta `.n8n`.
- 7 Recria os containers com Docker Compose.
- 8 Instala/configura Nginx e Certbot se necessário.
- 9 Valida HTTP local e HTTPS público.

Secrets esperados:

- 1 `VPS_HOST`
- 1 `VPS_USER`
- 1 `VPS_SSH_KEY`
- 1 `PRODUCTION_VPS_APP_DIR`
- 1 `N8N_HOST_PORT`
- 1 `POSTGRES_PASSWORD`
- 1 `OPENROUTER_API_KEY`
- 1 `N8N_BASIC_AUTH_USER`
- 1 `N8N_BASIC_AUTH_PASSWORD`
- 1 `N8N_ENCRYPTION_KEY`
- 1 `LETSENCRYPT_EMAIL`
- 1 `LANGSMITH_API_KEY`

Nginx

O exemplo em `infra/nginx/n8n.preambulo.cledson.com.br.conf` encaminha todo tráfego para o n8n local:

```
https://n8n.preambulo.cledson.com.br -> http://127.0.0.1:5678
```

O workflow de deploy também cria uma configuração Nginx equivalente e habilita TLS com Certbot.

Se quiser expor endpoints sem `/webhook`, é necessário adicionar uma regra explícita de rewrite/proxy. A configuração atual não faz essa remoção; portanto os endpoints padrão continuam em `/webhook/...`

Rotina operacional

Comandos úteis na VPS:

```
docker compose --env-file .env.production ps
docker compose --env-file .env.production logs -f n8n
docker compose --env-file .env.production logs -f postgres
docker compose --env-file .env.production pull
docker compose --env-file .env.production up -d --remove-orphans
```

Health check:

```
curl -i https://n8n.preambulo.cledson.com.br
curl -i https://n8n.preambulo.cledson.com.br/webhook/health
```

Backup

Para preservar uma instalação:

- 1 backup do volume `postgres-data`;
- 1 backup da pasta `.n8n`;
- 1 backup seguro dos secrets usados no deploy;
- 1 export periódico dos workflows atualizados pelo editor.

Atualização de workflows

Quando alterar workflows pelo editor n8n:

- 1 exporte o JSON atualizado;
- 2 substitua o arquivo correspondente em `workflows/`;
- 3 rode o script de validação;
- 4 versione a mudança no Git;
- 5 importe/ative no ambiente alvo quando necessário.

Validação e troubleshooting

Validadores

Os scripts PowerShell em `scripts/` verificam se os workflows exportados mantêm a estrutura esperada.

Rodar um validador:

```
powershell -ExecutionPolicy Bypass -File scripts/validate-review-workflow.ps1
```

Rodar todos:

```
Get-ChildItem scripts -Filter "validate-*.ps1" |  
ForEach-Object { powershell -ExecutionPolicy Bypass -File $_.FullName }
```

Validadores disponíveis:

- 1 `validate-review-workflow.ps1`
- 1 `validate-compliance-workflow.ps1`
- 1 `validate-document-workflow.ps1`
- 1 `validate-tests-workflow.ps1`
- 1 `validate-pull-request-review-workflow.ps1`
- 1 `validate-pull-request-tests-workflow.ps1`
- 1 `validate-history-list-workflow.ps1`
- 1 `validate-history-detail-workflow.ps1`
- 1 `validate-analytics-usage-workflow.ps1`
- 1 `validate-batch-workflow.ps1`
- 1 `validate-deploy-workflow.ps1`

Problemas comuns

Endpoint retorna 404

Confirme:

- 1 o workflow foi importado;

- 1 o workflow esta ativo;
- 1 a URL usa `/webhook/...` em producao;
- 1 durante teste manual, a URL temporaria pode ser `/webhook-test/...`;
- 1 o path do Webhook node corresponde ao esperado.

Editor abre, mas webhook nao exige Basic Auth

O Basic Auth do n8n protege o editor. Os Webhook nodes exportados nao possuem autenticacao propria. Proteja `/webhook/*` via Nginx/gateway ou adicione auth diretamente nos nodes.

Erro em nodes PostgreSQL

Verifique:

- 1 credencial `case-n8n Postgres`;
- 1 host `postgres` quando estiver dentro do Docker Compose;
- 1 usuario, senha e banco iguais ao `.env`;
- 1 container `postgres` saudavel;
- 1 SQL bootstrap executado em `infra/sql/init`.

Erro em nodes OpenRouter

Verifique:

- 1 credencial `OpenRouter account`;
- 1 chave `OPENROUTER_API_KEY`;
- 1 modelo enviado no payload;
- 1 disponibilidade do modelo no provedor;
- 1 logs do node LangChain/OpenRouter no editor n8n.

Fluxos de PR falham

Verifique:

- 1 `github_pull_request_url` no formato `https://github.com/org/repo/pull/123`;
- 1 credencial `GitHub account`;
- 1 permissao para ler PR, diff e arquivos;
- 1 limite de tamanho do diff;

- 1 se Jira foi enviado, `jira_base_url` precisa ser HTTPS e a credencial `Jira account` deve estar configurada.

Batch retorna erro nos itens

O batch chama os fluxos individuais por HTTP interno usando `http://127.0.0.1:5678/webhook/api/v1/...`

Confirme:

- 1 workflows individuais ativos;
- 1 paths sem rewrite inesperado;
- 1 payload de cada item valido para o fluxo;
- 1 `continue_on_error` conforme comportamento desejado.

History sem dados

Os dados aparecem depois que algum fluxo cria execucao em `executions`. Se apenas o `health` foi chamado, nao ha execucao persistida. Rode `review`, `tests`, `document`, `compliance` ou os fluxos de PR e consulte novamente.

Analytics zerado

Analytics agrega `executions` e `execution_telemetry`. Se as execucoes vierem somente de cache antigo sem telemetry, alguns campos podem ficar zerados. Gere uma nova execucao sem cache para validar.

Checklist rapido pos-importacao

- 1 Abrir editor n8n.
- 2 Confirmar credenciais.
- 3 Importar workflows.
- 4 Salvar e ativar workflows.
- 5 Chamar `/webhook/health`.
- 6 Chamar `/webhook/api/v1/review` com payload pequeno.
- 7 Consultar `/webhook/api/v1/history`.
- 8 Consultar `/webhook/api/v1/analytics/usage`.
- 9 Rodar validadores antes de versionar mudancas.